



MULTICORE/MANY-CORE
SYSTEMS-ON-CHIP

17th IEEE International Symposium on Embedded
Multicore/Manycore SoCs (MCSOC 2024)

Predictive Modeling for Thread Optimization in OpenMP-Based Parallelization Using Machine Learning

Presented By: Dr. Mushtaq Ahmed

Authors: Akash Yadav, Mushtaq Ahmed

December 18th, 2024



INTRODUCTION

- High-performance applications frequently use parallelization tools like OpenMP to achieve faster execution times.
- Programmers used Parametric and non-parametric approaches for program optimization.
- Parametric approaches involve selecting specific parameters, such as array padding and the number of threads, followed by predicting values that can highly optimize the programs.
- Non-parametric approaches focus on determining the optimal passes and their implementation order, which are interdependent and influence each other's effectiveness in optimizing the program.

INTRODUCTION

- Parameter prediction refers to the process of forecasting the values of specific variables that affect the execution and efficiency of programs and systems.
- These parameters can include execution time, memory usage, and hardware configuration settings, among others.
- Accurate parameter prediction can significantly enhance performance, reduce energy consumption, and improve resource management.
- While recent research in automated performance tuning has proposed run-time application configuration and compilation, most automatic compilers do not support such features.
- Traditional compiler optimization methods rely on expert-tuned heuristics.

PRESENTATION OUTLINE

- Proposed Approach
- Random Forest
- Dataset Generation
- Experimental Setup
- Result Analysis
- Conclusions & Future Work

OpenMP

- OpenMP uses the fork-join model to provide multi-threading-based parallelism.
- A master thread creates multiple slave threads that can work in parallel.
- Figure 1 shows an example of parallelizing a loop using four threads. The master thread (green colour) executes the program sequentially.
- When a parallel region occurs (`#pragma omp parallel for num_threads(4)`), master threads create three slave threads and distribute the loop iterations between them (including itself) to run the loop in parallel.

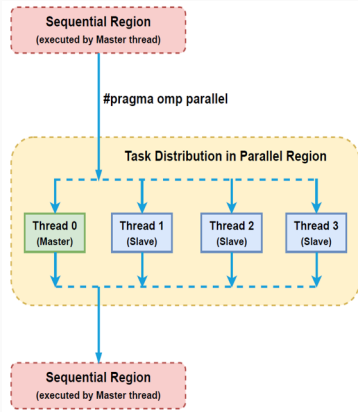


Figure 1: Parallelization using OpenMP

PROPOSED APPROACH

- The proposed approach uses the Random Forest Regression model to predict the optimal number of threads.
- The proposed method can be divided into three major phases:
 - Dataset generation and pre-processing phase
 - Model training phase
 - Model testing phase

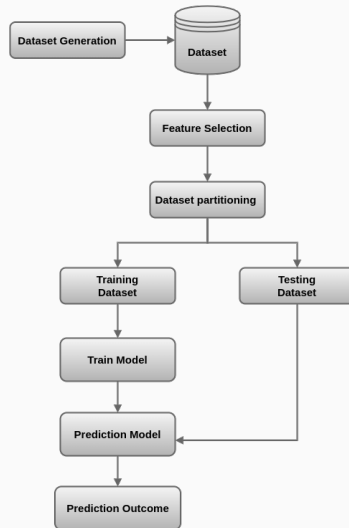


Figure 2: Flow diagram of proposed approach

RANDOM FOREST REGRESSION

- Random Forest (RF) is a versatile ensemble learning algorithm based on Decision Trees (DT).
- It builds multiple decision trees that involve randomly sampling the original dataset with replacement to create sub-datasets.
- Each tree is constructed without pruning and makes its prediction based on a subset of randomly selected input features.
- The final prediction in RF regression is obtained by averaging the predictions of all individual trees.

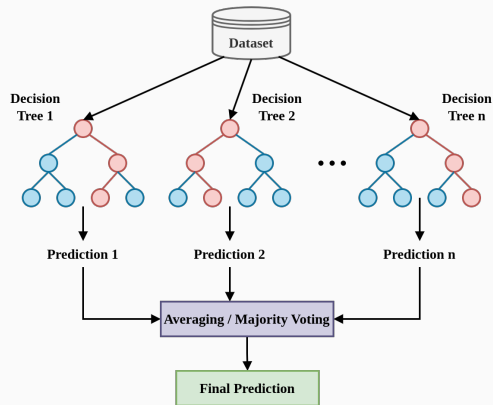


Figure 3: Random Forest Regression Model

DATASET GENERATION

- The Python-based loop generator program is used to create a variety of parallelizable loops, each containing different types of array-based operations.
- These operations are fundamental in various computational tasks and can significantly benefit from parallel execution.
- The program generates loops for operations such as:
 - **Copying a 1D array:** $B[i] = A[i]$.
 - **Multiplying each element by a scalar:** $A[i] = A[i] * C$
 - **Element-wise addition or multiplication of arrays:** $C[i] = A[i] + B[i]$ or $C[i] = A[i] * B[i]$

DATASET GENERATION

- Loops vary in complexity, from simple to multi-statement operations, simulating real-world workloads.
- This variety is intentional to simulate different computational workloads that might be encountered in practical applications.
- To evaluate the performance of these loops, they are executed in parallel using OpenMP directives using OpenMP with thread counts like 2, 4, 8, etc.
- Each configuration is executed five times to ensure reliability.
- The average execution time is recorded as the final result.

DATASET FEATURES

- Based on the parameters used for parallelization and the types of operations within the loops, the dataset is created to facilitate analysis.

Table 1: Description of Dataset Features

Sr. No.	Feature	Description	Value Type	Examples
1	nThread	Number of threads used.	Integer	2, 4
2	nState	Number of statements in the loop	Integer	1, 2
3	typeOp	Type of operations	Text	add, sub
4	numOp	Number of operations	Integer	1, 2
5	numVar	Number of Variable	Integer	2, 3
6	varType	Type of Variable	Text	int, float
7	numArray	Number of Arrays	Integer	1, 2
8	itr	Trip count of the loop.	Integer	100, 2000
9	exeTime	Loop execution time	Decimal	153.2, 78.1

DATASET FEATURES

- **nThread:** The number of threads impacts parallel execution, with more threads typically reducing execution time, although too many may introduce overhead.
- **nState:** The number of statements reflects loop complexity, where a higher count can increase execution duration.
- **typeOp:** Type of operations varies in computational cost,
- **numOp:** The number of operations correlates directly with runtime; more operations usually lead to longer execution times.
- **numVar & varType:** The number and type of variables affect memory usage and processing efficiency.
- **numArray & itr:** Additionally, the number of arrays influences memory access patterns, and the trip count determines loop iterations, directly affecting execution time.

EXPERIMENTAL SETUP

- The model was trained and evaluated on the Google Colab platform.
- Table 2 shows the various resources and libraries used in the training and testing of the model.
- The dataset was divided into training and testing sets with an 80:20 split.

Table 2: Specifications of Hardware and Software Packages Used for Model Training

Sr. No.	Item	Specification
1	Platform	Google Colab
2	Processor	T4 GPU
3	RAM	12 GB
4	Disk	78 GB
5	sklearn package	v1.2.2
6	pandas package	v2.0.3
7	matplotlib package	v3.7.1
8	numpy package	v1.25.2

EVALUATION METRICS

- In this study, the following evaluation metrics have been used for performance evaluation:
 - Root Mean Squared Error (RMSE)
 - R-Square (R^2)
 - Mean Absolute Percentage Error (MAPE)
- Notations used in the metrics equation are:
 - N denotes the sample size of the dataset,
 - y denotes the true output values,
 - $\hat{y}$ signifies the values predicted by the model, and
 - item k is an iterating parameter.

EVALUATION METRICS: RMSE

- RMSE is the square root of MSE, maintaining its differential properties, as given in Equation 1:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N (y_k - \hat{y}_k)^2} \quad (1)$$

- By taking the square root, RMSE addresses the issue of over-penalizing small errors inherent in MSE.

Table 3: Example of RMSE calculation

Actual (y)	Predicted ($\hat{y}$)	$(y - \hat{y})^2$	MSE	RMSE
20	22	4	8.2	2.86
30	28	4		
35	33	4		
40	42	4		
45	40	25		

EVALUATION METRICS: R^2

- R^2 is calculated using other metrics, as shown in Equation 2. It measures the proportion of variance in the target variable captured by the model.

$$R^2 = 1 - \frac{SSE}{SSTO} \quad (2)$$

$$SSE = \sum_{k=1}^N (y_k - \hat{y}_k)^2 \quad (3)$$

$$SSTO = \sum_{k=1}^N (y_k - \bar{y})^2 \quad (4)$$

- An R^2 value close to 1 indicates that the model captures nearly all the variance in the target variable, while a value close to 0 indicates that it captures none.

Table 4: Example of R^2 calculation

Actual (y)	Predicted ($\hat{y}$)	SSE	$SSTO$	R^2
20	22	4	196	0.89
30	28	4	16	
35	33	4	1	
40	42	4	36	
45	40	25	121	
Total		41	370	

EVALUATION METRICS: MAPE

- MAPE, also known as Mean Absolute Percentage Deviation (MAPD), is a measure of prediction accuracy.

- MAPE calculated as follows:

$$\text{MAPE}(y, \hat{y}) = \frac{1}{N} \sum_{k=0}^{N-1} \frac{y_k - \hat{y}_k}{y_k} \times 100 \quad (5)$$

- It is commonly used as a loss function for regression problems and in model evaluation.

Table 5: Example of MAPE calculation

Actual (y)	Predicted ($\hat{y}$)	APE
20	22	$\left \frac{20-22}{20} \right \times 100 = 10\%$
30	28	$\left \frac{30-28}{30} \right \times 100 = 6.67\%$
35	33	$\left \frac{35-33}{35} \right \times 100 = 5.71\%$
40	42	$\left \frac{40-42}{40} \right \times 100 = 5\%$
45	40	$\left \frac{45-40}{45} \right \times 100 = 11.11\%$
MAPE		7.69%

RESULT ANALYSIS

- The model results achieved are very good, demonstrating high prediction accuracy. The difference between the actual values and the predicted values of the test dataset is shown in Figure 4.

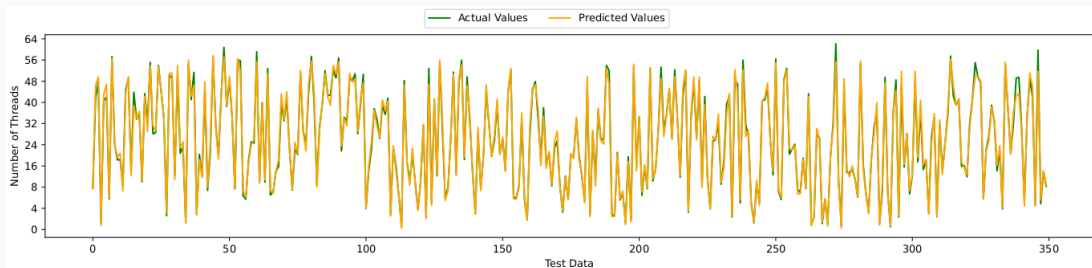


Figure 4: Comparison of actual thread count and predicted thread count by the proposed model for various test cases

RESULT ANALYSIS

- Figure 5 provides a zoomed-in view of Figure 4, showing the prediction results for only thirty test cases.
- From the graph, it can be stated that the proposed model accuracy is high and predicts the thread number near the actual numbers.

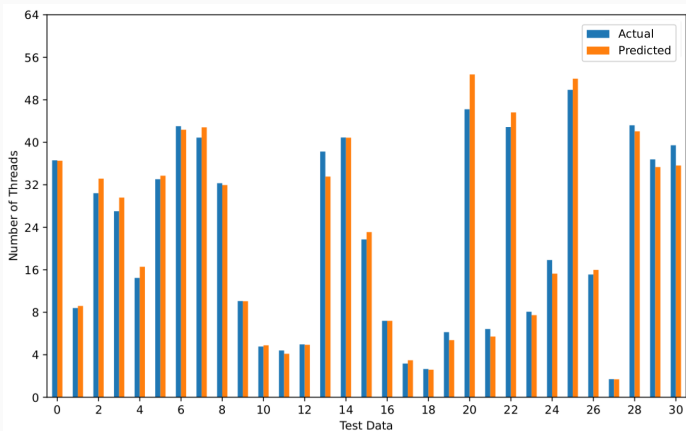


Figure 5: Detailed view of actual thread counts and predicted thread counts for 30 test cases

RESULT ANALYSIS

- In past research, ML techniques have been used to optimize various parameters of OpenMP directives, such as thread scheduling, reduction, and loop unrolling, but determining the number of optimal threads has received less attention.
- To evaluate the performance of the proposed model, it is also compared with other regression models using the evaluation metrics.

Table 6: Performance comparison of the proposed approach with other models

Model	R^2	MAPE	RMSE
Ridge Regression	0.9712	5.854	19.23
SDG Regression	0.9852	4.453	14.32
Decision Tree Regression	0.9758	5.352	17.68
Proposed Method	0.9901	3.253	11.54

RESULT ANALYSIS

- From the results, it is clear that random forest regression is the most accurate, evidenced by the lowest MAE and RMSE, and the highest R^2 value, making it highly effective for predicting the optimal number of threads for OpenMP directives.
- Both ridge regression and SGD models also show good performance with very similar metrics to each other, indicating their robustness and reliability in prediction tasks.
- The decision tree model has higher error rates, indicating that it might not capture the underlying data patterns as effectively as the other models.
- Overall, The ensemble feature of random forest, which combines multiple decision trees, significantly enhances its predictive accuracy and stability, making it the best choice among the evaluated models.

CONCLUSIONS & FUTURE WORK

- A Random Forest Regression-based machine learning technique was used to predict the optimal number of threads for parallelizing loops with OpenMP directives.
- The Random Forest model outperformed other regression methods, achieving the lowest MAPE, RMSE, and the highest R^2 , demonstrating superior accuracy and robustness.
- This automation significantly enhances the efficiency of parallelized code execution in high-performance computing.
- The future work involves exploring advanced machine learning techniques, particularly deep learning, to validate the model.
- Assessing the model's scalability using larger, more complex datasets.

REFERENCES

- Jinying Kong, Kai Nie, Qinglei Zhou, Jinlong Xu, and Lin Han. Thread private variable access optimization technique for Sunway high-performance multi-core processors. In International Conference of Pioneering Computer Scientists, Engineers and Educators, pages 180–189. Springer, 2021.
- Hui Liu, Jinlong Xu, Sen Chen, and Te Guo. Compiler optimization parameter selection method based on ensemble learning. *Electronics*, 11(15), 2022.
- Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '19, pages 2623–2631, New York, NY, USA, 2019.
- Saman Amarasinghe. Compiler 2.0: Using machine learning to modernize compiler technology, pages 1–2, 2020.
- Engin Kayraklioglu, Erwan Favry, and Tarek El-Ghazawi. A machine learning approach for productive data locality exploitation in parallel computing systems. In 2019 19th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing (CCGRID), pages 361–370, 2019.
- Raphael Mosaner, Gergo Barany, David Leopoldseder, and Hanspeter Mössenböck. Improving vectorization heuristics in a dynamic compiler with machine learning models. In Proceedings of the 14th ACM SIGPLAN International Workshop on Virtual Machines and Intermediate Languages, VMIL 2022, pages 36–47, New York, NY, USA, 2022.
- Le Chen, Quazi Ishtiaque Mahmud, Hung Phan, Nesreen K. Ahmed, and Ali Jannesari. Learning to parallelize with OpenMP by augmented heterogeneous AST representation, 2023.

REFERENCES

- Alessio Colucci, David Juhász, Martin Mosbeck, Alberto Marchisio, Semeen Rehman, Manfred Kreutzer, Gunther Nadbath, Axel Jantsch, and Muhammad Shafique. MLCOMP: A methodology for machine learning-based performance estimation and adaptive selection of Pareto-optimal compiler optimization sequences. In 2021 Design, Automation & Test in Europe Conference & Exhibition (DATE),
- Kai Nie, Qinglei Zhou, Hong Qian, Jianmin Pang, Jinlong Xu, and Yapeng Li. Parallel region reconstruction technique for Sunway high-performance multi-core processors. In Data Science: 7th International Conference of Pioneering Computer Scientists, Engineers and Educators, ICPCSEE 2021, Taiyuan, China, September 17–20, 2021, Proceedings, Part I, pages 163–179. Springer, 2021.
- Dheya Mustafa. A survey of performance tuning techniques and tools for parallel applications. IEEE Access, 10:15036–15055, 2022.
- Georgios Zacharopoulos, Andrea Barbon, Giovanni Ansaloni, and Laura Pozzi. Machine learning approach for loop unrolling factor prediction in high-level synthesis, July 2018.
- Hui Liu and Zhanjie Guo. A loop unrolling method based on machine learning. Vibroengineering PROCEDIA, 18:215–221, May 2018. Hui Liu, Jinlong Xu, Sen Chen, and Te Guo. Compiler optimization parameter selection method based on ensemble learning. Electronics, 11(15), 2022.
- Yang Liu, Wissam M. Sid-Lakhdar, Osni Marques, Xinran Zhu, Chang Meng, James W. Demmel, and Xiaoye S. Li. GPTune: Multitask learning for autotuning exascale applications. In Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, PPOPP '21, pages 234–246, New York, NY, USA, 2021.



MULTICORE/MANY-CORE
SYSTEMS-ON-CHIP

17th IEEE International Symposium on Embedded
Multicore/Manycore SoCs (MCSoc 2024)

THANK YOU

